STAT 432 Homework 01

Conner Jensen (connerj2)

Contents

Question 1 (Least squares and matrix calculations) **2**

(a) . . . 2

(b) . . . 2

(c) . . . 3

Question 2 (Gaussian likelihood for linear regression) **3**

(a) . . . 3

(b) . . . 3

(c) . . . 4

Question 3 (Starting values in nonconvex optimization) **5**

(a) . . . 5

(b) . . . 6

(c) . . . 6

Question 4 (Nearly collinear predictors) **6**

(a) . . . 6

(b) . . . 7

(c) . . . 8

Question 5 (Data preparation and summary statistics) **8**

(a) . . . 8

(b) . . . 9

(c) . . . 9

Question 6 (Can 39 million Fitbit records represent US adults?) **9**

(a) . . . 9

(b) . . . 10

(c) . . . 10

Question 1 (Least squares and matrix calculations)

Set the random seed to 43201. Generate $n = 100$ observations and $p = 5$ predictors from

$$x_{ij} \overset{\text{iid}}{\sim} N(0,1), \qquad \varepsilon_i \overset{\text{iid}}{\sim} N(0, 0.7^2),$$

independently. Form $\mathbf{X} = [\mathbf{1}, \mathbf{x}_1, \ldots, \mathbf{x}_5]$ and

$$\boldsymbol{\beta} = (3, 1.4, -0.9, 0.7, 0, 1.1)^\top, \qquad \mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}.$$

(a)

The least-squares estimate is obtained from the QR factorization of $\mathbf{X}$ via `qr.coef()`, so $(\mathbf{X}^\top\mathbf{X})^{-1}$ is never formed.

```
set.seed(43201)
n <- 100
p <- 5
X <- cbind(1, matrix(rnorm(n * p), n, p))
beta <- c(3, 1.4, -0.9, 0.7, 0, 1.1)
y <- as.numeric(X %*% beta + rnorm(n, sd = 0.7))

dim(X)
```

```
## [1] 100   6
```

```
qr(X)$rank
```

```
## [1] 6
```

```
beta_hat <- qr.coef(qr(X), y)
rbind(true = beta, estimate = as.numeric(beta_hat))
```

```
##              [,1]    [,2]       [,3]      [,4]      [,5]     [,6]
## true     3.000000 1.40000 -0.9000000 0.7000000 0.0000000 1.100000
## estimate 3.006321 1.41716 -0.8883453 0.7952834 0.0484378 1.131934
```

$\mathbf{X}$ is 100×6 and has full rank 6. The estimates are close to the true coefficients. The intercept and the nonzero slopes recover the signs and magnitudes used to generate the data; the coefficient of x_4 (true value 0) is a small nonzero number, which is expected with $n = 100$ and $\sigma = 0.7$.

(b)

```
yhat <- as.numeric(X %*% beta_hat)
r <- y - yhat
rmse <- sqrt(mean(r^2))
rmse
```

```
## [1] 0.6763696
```

The training RMSE is about 0.676, close to the true error standard deviation 0.7. That is the typical size of a residual, not a statement about bias in $\widehat{\boldsymbol{\beta}}$.

(c)

```
max(abs(crossprod(X, r)))
```

```
## [1] 9.689703e-14
```

$\|\mathbf{X}^\mathsf{T}\mathbf{r}\|_\infty$ is on the order of 10^{-13}. The normal equations $\mathbf{X}^\mathsf{T}\mathbf{X}\widehat{\boldsymbol{\beta}} = \mathbf{X}^\mathsf{T}\mathbf{y}$ are equivalent to $\mathbf{X}^\mathsf{T}\mathbf{r} = \mathbf{0}$. So the residual is numerically orthogonal to every column of $\mathbf{X}$, including the intercept. The tiny leftover is floating-point error, not a failure of least squares.

Question 2 (Gaussian likelihood for linear regression)

Continue with the same $\mathbf{X}$ and $\mathbf{y}$. The model is $\mathbf{Y} \mid \mathbf{X} \sim N_n(\mathbf{X}\boldsymbol{\beta}, \sigma^2\mathbf{I}_n)$, with log-likelihood

$$\ell(\boldsymbol{\beta}, \sigma^2) = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}(\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^\mathsf{T}(\mathbf{y} - \mathbf{X}\boldsymbol{\beta}).$$

(a)

For fixed σ^2, the first term does not depend on $\boldsymbol{\beta}$. The second term is a negative constant times the residual sum of squares $\mathrm{RSS}(\boldsymbol{\beta}) = (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^\mathsf{T}(\mathbf{y} - \mathbf{X}\boldsymbol{\beta})$. Maximizing ℓ in $\boldsymbol{\beta}$ is therefore the same as minimizing $\mathrm{RSS}(\boldsymbol{\beta})$. The MLE $\widehat{\boldsymbol{\beta}}_{\mathrm{MLE}}$ is the ordinary least-squares estimator from Question 1.

(b)

Write $s = \sigma^2$ and $\mathrm{RSS} = (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^\mathsf{T}(\mathbf{y} - \mathbf{X}\boldsymbol{\beta})$. Then

$$\ell(\boldsymbol{\beta}, s) = -\frac{n}{2}\log(2\pi s) - \frac{\mathrm{RSS}}{2s}.$$

Differentiating in s,

$$\frac{\partial \ell}{\partial s} = -\frac{n}{2s} + \frac{\mathrm{RSS}}{2s^2}.$$

Setting the derivative to zero gives $\widehat{\sigma}^2_{\mathrm{MLE}} = \mathrm{RSS}/n$. The usual unbiased estimator uses denominator $n - k$, where $k = 6$ is the number of columns of $\mathbf{X}$:

$$\widehat{\sigma}^2_{\mathrm{unb}} = \frac{\mathrm{RSS}}{n - 6}.$$

```
rss <- sum(r^2)
k <- ncol(X)
sigma2_mle <- rss / n
sigma2_unb <- rss / (n - k)
c(MLE = sigma2_mle, unbiased = sigma2_unb)
```

```
##       MLE  unbiased
## 0.4574759 0.4866765
```

The MLE divides by n because that is what the score equation produces. It does not correct for the k degrees of freedom already used to estimate $\boldsymbol{\beta}$, so it is biased downward. The unbiased estimator uses $n - k$ for that reason.

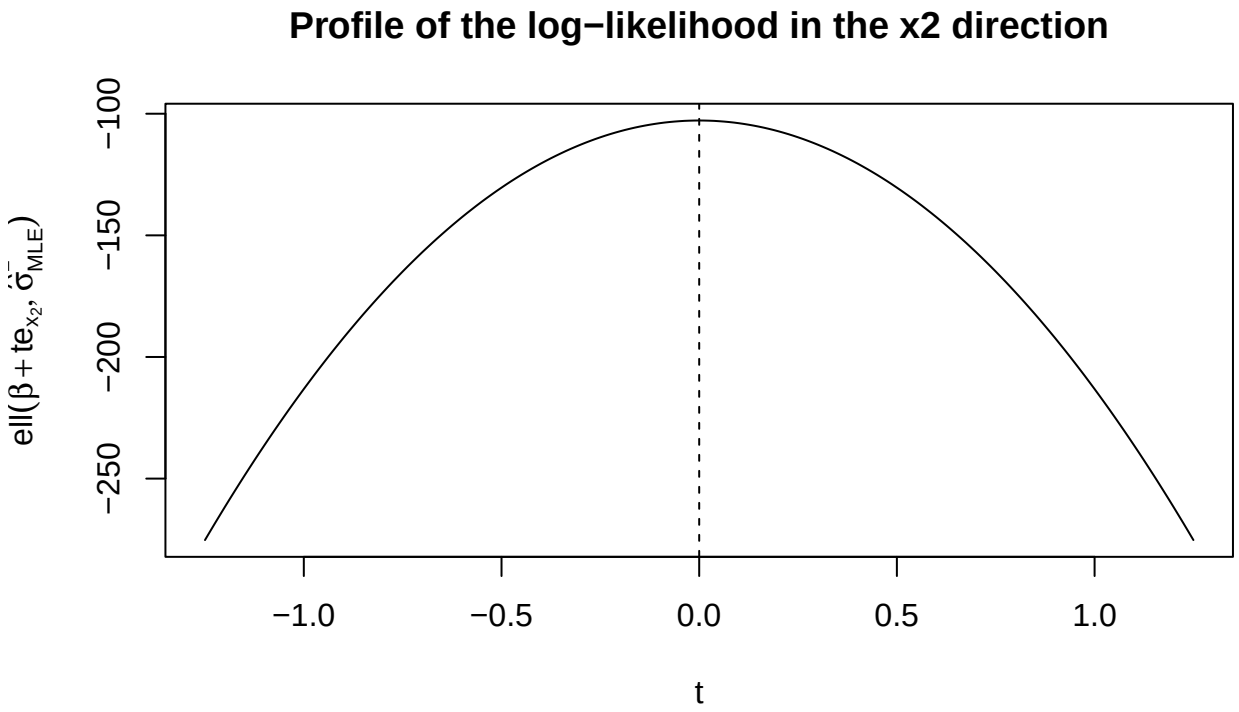
(c)

Perturb only the coefficient of x_2 by adding $t\mathbf{e}_{x_2}$, with $\mathbf{e}_{x_2} = (0, 0, 1, 0, 0, 0)^\mathsf{T}$.

```
loglik <- function(beta, sigma2) {
  resid <- y - as.numeric(X %*% beta)
  -n / 2 * log(2 * pi * sigma2) - sum(resid^2) / (2 * sigma2)
}

e_x2 <- c(0, 0, 1, 0, 0, 0)
t_grid <- seq(-1.25, 1.25, length.out = 201)
ell_t <- sapply(t_grid, function(t) {
  loglik(beta_hat + t * e_x2, sigma2_mle)
})

plot(t_grid, ell_t, type = "l",
     xlab = expression(t),
     ylab = expression(ell(hat(beta) + t * e[x[2]], hat(sigma)[MLE]^2)),
     main = "Profile of the log-likelihood in the x2 direction")
abline(v = 0, lty = 2)
```



```r
t_grid[which.max(ell_t)]
```

```
## [1] 0
```

The curve is maximized at $t = 0$. Moving away from $\widehat{\boldsymbol{\beta}}$ increases RSS and therefore decreases the Gaussian log-likelihood. That matches the least-squares result: $\widehat{\boldsymbol{\beta}}$ already minimizes RSS, so it also maximizes $\ell(\boldsymbol{\beta}, \widehat{\sigma}^2_{\text{MLE}})$.

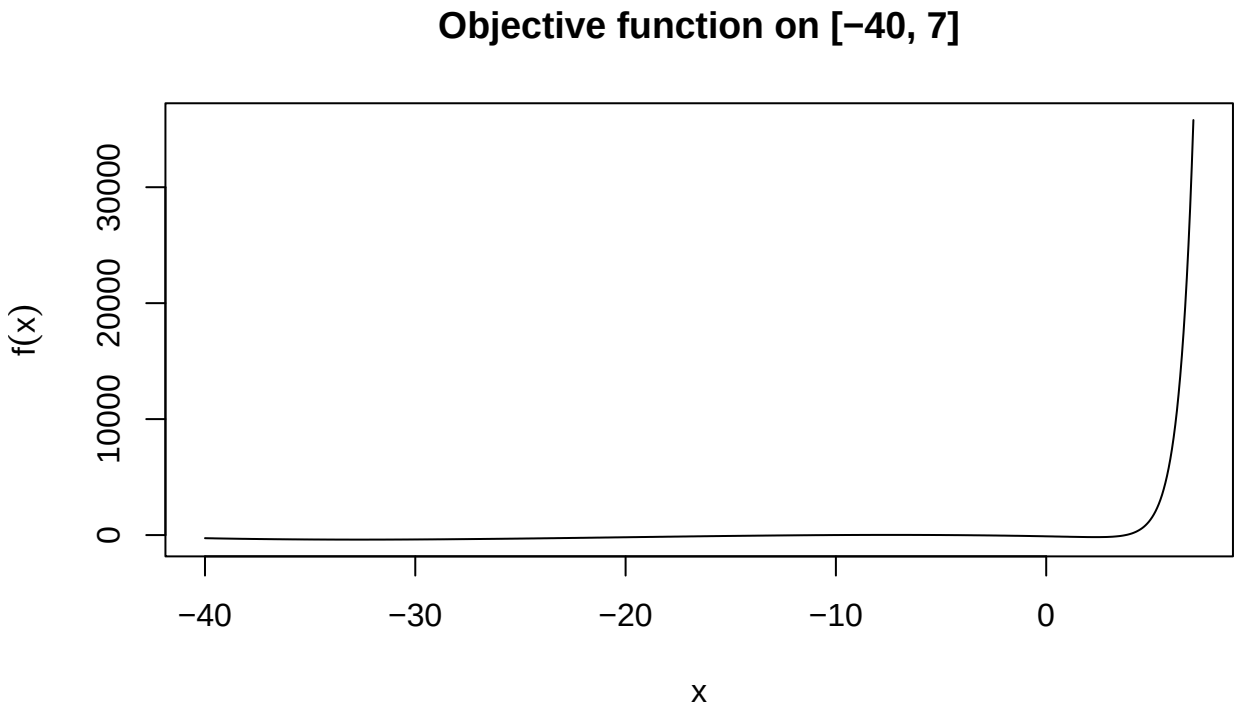
Question 3 (Starting values in nonconvex optimization)

$$f(x) = \exp(1.5x) - 3(x+6)^2 - 0.05x^3, \qquad f'(x) = 1.5\exp(1.5x) - 6(x+6) - 0.15x^2.$$

(a)

```r
f <- function(x) exp(1.5 * x) - 3 * (x + 6)^2 - 0.05 * x^3
fp <- function(x) 1.5 * exp(1.5 * x) - 6 * (x + 6) - 0.15 * x^2

x_plot <- seq(-40, 7, length.out = 600)
plot(x_plot, f(x_plot), type = "l",
     xlab = expression(x), ylab = expression(f(x)),
     main = "Objective function on [-40, 7]")
```



The function is not convex. There is a deep valley near $x \approx -33$ and a shallower basin near $x \approx 2$. A local optimizer that only uses f and f' can stop in either basin. The exponential term also grows quickly for large positive x, so the right side of the interval is steep.

(b)

```
run_bfgs <- function(x0) {
  opt <- optim(par = x0, fn = f, gr = fp, method = "BFGS")
  data.frame(
    start = x0,
    x = opt$par,
    f = opt$value,
    abs_fprime = abs(fp(opt$par)),
    converged = opt$convergence == 0
  )
}

rbind(run_bfgs(-15), run_bfgs(0))
```

```
##   start          x         f   abs_fprime converged
## 1   -15 -32.649111 -390.3858 8.725087e-08      TRUE
## 2     0   2.349967 -175.8626 6.556057e-07      TRUE
```

Both runs report convergence. Starting at -15 ends near $x \approx -32.65$ with $f \approx -390.4$ and $|f'|$ about 10^{-7}. Starting at 0 ends near $x \approx 2.35$ with $f \approx -175.9$ and $|f'|$ about 10^{-6}.

(c)

The two starts lie in different basins, so BFGS follows the local gradient into two different stationary points. The run from -15 has the lower objective value.

A single local run is not enough to call that point the global minimum. One would need a broader search (many starts, a grid, or a global method) or a mathematical argument that no smaller value exists on $\mathbb{R}$. A grid over $[-40, 7]$ also points to the left-hand valley, which is supporting evidence on this interval, but it is still not a proof for the whole real line.

Question 4 (Nearly collinear predictors)

Continue with $\mathbf{X}$ and $\mathbf{y}$ from Question 1. With seed 43202,

$$\mathbf{x}_6 = \mathbf{x}_1 + 10^{-4}\mathbf{u}, \qquad \mathbf{X}_+ = [\mathbf{X}, \mathbf{x}_6], \qquad \mathbf{y}^\star = \mathbf{y} + 10^{-3}\mathbf{u},$$

so that $\mathbf{y}^\star - \mathbf{y} = 10(\mathbf{x}_6 - \mathbf{x}_1)$.

(a)

```r
set.seed(43202)
u <- rnorm(n)
x1 <- X[, 2]
x6 <- x1 + 1e-4 * u
Xplus <- cbind(X, x6)
ystar <- y + 1e-3 * u

fit_X <- qr.coef(qr(X), y)
fit_Xp <- qr.coef(qr(Xplus), y)

cond_rmse <- function(XX, yy, bb) {
  c(condition = kappa(XX, exact = TRUE),
    RMSE = sqrt(mean((yy - as.numeric(XX %*% bb))^2)))
}

rbind(
  X = cond_rmse(X, y, fit_X),
  Xplus = cond_rmse(Xplus, y, fit_Xp)
)
```

```
##          condition      RMSE
## X         1.524914 0.6763696
## Xplus 20209.203501 0.6754668
```

```r
c(x1 = unname(fit_Xp[2]), x6 = unname(fit_Xp[7]))
```

```
##        x1        x6
##  351.6766 -350.2588
```

$\mathbf{X}$ is well conditioned. Adding $\mathbf{x}_6$ raises the condition number by about four orders of magnitude because $\mathbf{x}_6$ is almost a copy of $\mathbf{x}_1$. Training RMSE barely changes: the extra column does not improve the fit in any useful way. The two collinear coefficients become huge and opposite, about 352 and -350.

(b)

```r
fit_star <- qr.coef(qr(Xplus), ystar)
delta_coef <- c(x1 = unname(fit_star[2] - fit_Xp[2]),
                x6 = unname(fit_star[7] - fit_Xp[7]))
delta_coef
```

```
##  x1  x6
## -10  10
```

```r
yhat_p <- as.numeric(Xplus %*% fit_Xp)
yhat_star <- as.numeric(Xplus %*% fit_star)
c(rmsd = sqrt(mean((yhat_star - yhat_p)^2)),
  max_abs = max(abs(yhat_star - yhat_p)))
```

```
##        rmsd     max_abs
## 0.001015879 0.002572675
```

The x_1 coefficient drops by 10 and the x_6 coefficient rises by 10. The fitted vectors differ by only about 0.001 in RMS and about 0.003 at most.

(c)

The identity $\mathbf{y}^\star - \mathbf{y} = 10(\mathbf{x}_6 - \mathbf{x}_1)$ says that the change in the response lies exactly in the direction that trades $\mathbf{x}_1$ for $\mathbf{x}_6$. Shifting the two coefficients by $(-10, 10)$ produces

$$-10\mathbf{x}_1 + 10\mathbf{x}_6 = 10(\mathbf{x}_6 - \mathbf{x}_1) = \mathbf{y}^\star - \mathbf{y}.$$

The fitted values therefore absorb the tiny perturbation in $\mathbf{y}$ while the separate coefficients jump by 10. Near-collinear predictors do not have separately interpretable effects: many coefficient pairs give almost the same fit, and a small change in $\mathbf{y}$ can move a large amount of weight from one variable to the other.

Question 5 (Data preparation and summary statistics)

Missing responses are dropped. They are not filled with 0 or a group mean.

(a)

```
raw <- read.csv("data/data-manipulation.csv", stringsAsFactors = FALSE)

dim(raw)
```

```
## [1] 24  5
```

```
sapply(raw, class)
```

```
## observation_id          group             x1             x2       response
##    "character"    "character"      "numeric"      "numeric"      "numeric"
```

```
sum(duplicated(raw$observation_id))
```

```
## [1] 0
```

```
colSums(is.na(raw))
```

```
## observation_id          group             x1             x2       response
##              0              0              0              0              3
```

```
analysis <- subset(raw, !is.na(response))
analysis$feature_sum <- analysis$x1 + analysis$x2
```

The file has 24 rows and 5 columns. `observation_id` and `group` are character; `x1`, `x2`, and `response` are numeric. Identifiers are unique. Three responses are missing (`obs-04`, `obs-11`, `obs-20`); the other columns are complete. The analysis table keeps the 21 rows with a recorded response and adds `feature_sum = x1 + x2`.

(b)

```
do.call(rbind, lapply(split(analysis, analysis$group), function(d) {
  data.frame(
    group = d$group[1],
    n = nrow(d),
    mean_response = mean(d$response),
    mean_feature_sum = mean(d$feature_sum),
    row.names = NULL
  )
}))
```

```
##   group n mean_response mean_feature_sum
## A     A 7      5.600000         3.600000
## B     B 7      7.602857         4.928571
## C     C 7      7.997143         6.585714
```

These summaries use only the 21 observations with a nonmissing response, split by group. They do not include the three rows whose response was missing.

(c)

```
sorted <- analysis[order(analysis$response, decreasing = TRUE), ]
sorted[1:3, c("observation_id", "group", "response", "feature_sum")]
```

```
##    observation_id group response feature_sum
## 22         obs-22     C     9.24         6.6
## 16         obs-16     B     9.00         5.2
## 14         obs-14     B     8.66         5.0
```

```
stopifnot(nrow(analysis) == sum(!is.na(raw$response)))
stopifnot(!any(is.na(analysis$response)))
stopifnot(!any(duplicated(analysis$observation_id)))
```

The three largest responses are `obs-22` (group C, 9.24), `obs-16` (group B, 9.00), and `obs-14` (group B, 8.66). The checks confirm that the analysis table has exactly as many rows as nonmissing responses in the original file, no missing responses, and unique identifiers.

Question 6 (Can 39 million Fitbit records represent US adults?)

The target is the mean, over US adults, of each adult's average daily step count during the study period, with equal weight on each adult.

(a)

The 39 million daily records are not independent. Days from the same person are repeated measurements: they share that person's routine, health, work schedule, and willingness to wear the device. People who

wear the tracker more often, stay in the study longer, or are more active may contribute many more days than people who drop the device or have gaps in syncing. Averaging every recorded day therefore weights high-volume participants more than low-volume participants. That is not the same as giving each adult equal weight.

(b)

The BYOD versus WEAR median comparison does not show that already owning a Fitbit causes people to walk more. Group membership is not randomized. BYOD participants already owned a device and chose to share data; WEAR participants were invited and given a device. Those routes select different people, which is already visible in the reported composition (for example, 77.3% versus 55.1% White, and 6.1% versus 15.2% with household income between $10,000 and $25,000). A plausible alternative is that people who already buy and use a Fitbit are, on average, more health-interested or more affluent, and would have higher step counts even without ownership itself causing the difference.

(c)

A large n does not fix selection. The participants are adults who enrolled in All of Us and either already had a Fitbit or accepted one, so they need not look like all US adults. The recorded days are a further selected subset: days with a synced, valid wear record. If more active, more adherent, or more advantaged people contribute more days, the naive average of all daily records mixes participant-level selection with day-level selection. That average can be far from the equal-weighted mean of person-specific averages in the US adult population, even with 39 million rows.

Source

Patten, T., Preble, E. A., Master, H., et al. (2026). The All of Us Research Program's wearables dataset. *Nature Medicine*, 32, 2302–2310.